

Toán số học & tổ hợp

0. Bài này dạy gì?

Ở Toán học cơ bản, bạn đã làm quen với ước số, GCD/LCM, số nguyên tố, sàng Eratosthenes, số học modular và lũy thừa nhanh. Bài Bậc này mở rộng chúng để xử các bài đếm/tổ hợp modulo p – một dạng đề rất phổ biến.

Bạn sẽ học:

1. Số học modular – nhắc lại + bấy thường gặp.
2. Phép chia mod là gì? – Nghịch đảo modulo & định lý Fermat.
3. Tổ hợp $C(n, k) \pmod p$ trong $O(1)$ sau tiền xử lý $O(n)$.
4. Sàng tuyến tính & Smallest Prime Factor (SPF).
5. Bù trừ (inclusion-exclusion) – đếm ít nhất một.

1. Số học modular – nhắc nhanh

Khi đề yêu cầu in đáp án modulo p (thường $p = 10^9 + 7$ hoặc 998244353, đều là số nguyên tố), ta giữ kết quả luôn ở dạng $r \pmod p \in [0, p)$. Tin vui:

PHÉP	CÔNG THỨC	VÍ DỤ ($P = 7$)
Cộng	$(a + b) \pmod p$	$(5 + 4) \pmod 7 = 2$
Trừ	$(a - b + p) \pmod p$	$(2 - 5 + 7) \pmod 7 = 4$
Nhân	$(a \cdot b) \pmod p$	$(3 \cdot 5) \pmod 7 = 1$

Cộng / trừ / nhân đều "phân phối" qua modulo, chỉ cần lấy mod ngay sau mỗi bước để tránh tràn số.

Bấy hay gặp với phép trừ (C++): $(a - b) \% p$ có thể ra số âm khi $a < b$. Luôn viết:

```
long long sub_mod(long long a, long long b, long long p) {  
    return (a - b + p) % p; // cong p de chac chan khong am  
}
```

Lũy thừa nhanh `power(a, b, p)` đã học – phần này coi như đã quen.

2. Vì sao "chia" lại khó?

Cộng / trừ / nhân ổn rồi – còn chia thì sao?

$$6 \pmod 5 = 1 \quad \text{và} \quad 2 \pmod 5 = 2$$

$$(6/2) \pmod 5 = 3 \quad \neq \quad (1/2) \pmod 5 = 0.5 \quad \text{Không phải số nguyên!}$$

Phép chia không "phân phối" qua modulo – đó là lý do ta cần một công cụ mới: nghịch đảo modulo.

Ý tưởng: thay vì viết $\frac{a}{b} \pmod{p}$, ta viết $a \cdot b^{-1} \pmod{p}$ trong đó b^{-1} là số nhân vào ra 1:

$$b \cdot b^{-1} \equiv 1 \pmod{p}$$

Khi đó $\frac{a}{b} \pmod{p} = a \cdot b^{-1} \pmod{p}$. Vậy bài toán quy về: tìm $b^{-1} \pmod{p}$.

3. Tìm nghịch đảo modulo

Định nghĩa: $a^{-1} \pmod{p}$ là số $x \in [1, p-1]$ thỏa $a \cdot x \equiv 1 \pmod{p}$.

Điều kiện tồn tại: $\gcd(a, p) = 1$. Khi p là số nguyên tố và $1 \leq a < p$ thì luôn tồn tại.

3.0. Cách "trâu bò": thử lần lượt

Với p nhỏ, ta thử $x = 1, 2, 3, \dots, p-1$ xem cái nào nhân với a ra 1 mod p :

Tìm $3^{-1} \pmod{7}$:

X	$3 \cdot X \pmod{7}$
1	3
2	6
3	2
4	5
5	1 ✓

$\rightarrow 3^{-1} \equiv 5 \pmod{7}$. Kiểm tra: $3 \cdot 5 = 15 = 2 \cdot 7 + 1$, đúng!

Cách này $O(p)$ – quá chậm với $p = 10^9 + 7$. Ta cần công cụ tốt hơn.

3.1. Bằng định lý Fermat (khi p nguyên tố)

$$a^{p-1} \equiv 1 \pmod{p}$$

Suy ra: $a \cdot a^{p-2} \equiv 1 \pmod{p}$, tức a^{p-2} chính là nghịch đảo của a :

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Áp dụng cho $a = 3, p = 7$: $3^{-1} \equiv 3^5 \pmod{7}$.

Tính từng bước: $3^2 = 9 \equiv 2, 3^4 = 2^2 = 4, 3^5 = 3^4 \cdot 3 = 4 \cdot 3 = 12 \equiv 5 \pmod{7}$ – khớp với cách trâu bò!

Tính bằng lũy thừa nhanh – $O(\log p)$ nhanh chóng mặt:

C++

```

long long power(long long a, long long b, long long p) {
    long long res = 1; a %= p;
    while (b > 0) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

long long inv(long long a, long long p) {
    return power(a, p - 2, p);
}

```

Python

```

def power(a, b, p):
    return pow(a, b, p) # pow(a, b, p) la built-in: O(log b)

def inv(a, p):
    return pow(a, p - 2, p) # hoac pow(a, -1, p) (Python 3.8+)

```

3.2. Khi p không nguyên tố – Extended Euclid

Khi p không nguyên tố (vẫn yêu cầu $\gcd(a, p) = 1$), Fermat không áp dụng được. Cách tổng quát là thuật toán Euclid mở rộng (extended Euclid): giải $ax + py = 1$, khi đó $x \bmod p$ chính là nghịch đảo.

Tuy nhiên, 99% bài CP dùng p nguyên tố (thường là $10^9 + 7$), Fermat là đủ.

4. Tổ hợp $C(n, k)$ – định nghĩa & tính chất

Định nghĩa: $C(n, k)$ (đọc là "tổ hợp chập k của n ", còn ký hiệu $\binom{n}{k}$) là số cách chọn k phần tử từ một tập n phần tử mà không quan tâm thứ tự.

Ví dụ: $C(4, 2) = 6$ – từ tập $\{A, B, C, D\}$ có 6 cách chọn 2 phần tử: $\{AB, AC, AD, BC, BD, CD\}$.

Công thức:

$$C(n, k) = \frac{n!}{k! \cdot (n - k)!} \quad (0 \leq k \leq n)$$

Quy ước $C(n, k) = 0$ khi $k < 0$ hoặc $k > n$.

Kiểm tra: $C(5, 2) = \frac{5!}{2! \cdot 3!} = \frac{120}{2 \cdot 6} = 10$ (Từ $\{1, 2, 3, 4, 5\}$ chọn 2 số: 10 cách).

Vài tính chất hay dùng:

- $C(n, 0) = C(n, n) = 1$
- $C(n, k) = C(n, n - k)$ (đối xứng – chọn k phần tử "vào" giống chọn $n - k$ phần tử "ra").

- $C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$ (tam giác Pascal).

Tam giác Pascal (mỗi số = tổng 2 số ngay trên):

$n = 0$: 1
 $n = 1$: 1 1
 $n = 2$: 1 2 1
 $n = 3$: 1 3 3 1
 $n = 4$: 1 4 6 4 1
 $n = 5$: 1 5 10 10 5 1

Hàng n , cột k chính là $C(n, k)$. Pascal cho ta cách tính $C(n, k)$ bằng DP $O(n^2)$ – đủ khi $n \leq 5000$. Khi n lớn hơn, ta cần cách $O(1)$ /truy vấn.

5. Tính $C(n, k) \bmod p$ trong $O(1)$ sau tiền xử lý $O(n)$

Modulo p , ta không chia trực tiếp được – phải nhân với nghịch đảo của giai thừa (mục 3). Tiền tính:

$$\begin{aligned} \text{fact}[i] &= i! \bmod p \\ \text{inv_fact}[i] &= (i!)^{-1} \bmod p \end{aligned}$$

Khi đó:

$$C(n, k) \bmod p = \text{fact}[n] \cdot \text{inv_fact}[k] \cdot \text{inv_fact}[n - k] \bmod p$$

Code chuẩn:

C++

```
const int N = 200005;
const long long MOD = 1000000007;

long long fact[N], inv_fact[N];

void precompute() {
    fact[0] = 1;
    for (int i = 1; i < N; i++) fact[i] = fact[i - 1] * i % MOD;
    inv_fact[N - 1] = power(fact[N - 1], MOD - 2, MOD);
    for (int i = N - 2; i >= 0; i--)
        inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD;
}

long long combi(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD;
}
```

Python

```
MOD = 1000000007
fact = [1] * N
for i in range(1, N):
    fact[i] = fact[i - 1] * i % MOD
```

```

inv_fact = [1] * N
inv_fact[N - 1] = pow(fact[N - 1], MOD - 2, MOD)
for i in range(N - 2, -1, -1):
    inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD

def combi(n, k):
    if k < 0 or k > n: return 0
    return fact[n] * inv_fact[k] % MOD * inv_fact[n - k] % MOD

```

Trick quan trọng – tại sao build `inv_fact` ngược? Cách "ngây thơ" là gọi Fermat cho từng `inv_fact[i] → O(n log p)`. Cách thông minh: tính `inv_fact[N-1]` một lần bằng Fermat, rồi truy hồi ngược:

$$\text{inv_fact}[i] = \text{inv_fact}[i + 1] \cdot (i + 1) \bmod p$$

Tại sao đúng? $(i + 1)! = (i + 1) \cdot i!$, lấy nghịch đảo cả hai vế: $\frac{1}{i!} = (i + 1) \cdot \frac{1}{(i+1)!}$, vậy toàn bộ tiền xử lý chỉ $O(n + \log p)$.

Bây: nhớ dùng `fact[n]` (KHÔNG phải `fact[k]`) ở thừa số đầu – nhầm vị trí là sai ngay.

Ứng dụng phổ biến:

- Đếm số cách chọn / sắp xếp.
- Đếm số đường đi trên lưới $n \times m$ (chỉ đi phải / xuống) $= C(n + m, n)$.
- Đếm số dãy thỏa điều kiện tổ hợp.

6. Sàng tuyến tính & Smallest Prime Factor (SPF)

Sàng tuyến tính $O(n)$ có một lợi thế lớn: nó tự nhiên cho ta ước nguyên tố nhỏ nhất (smallest prime factor, SPF) của mỗi số, giúp phân tích thừa số nguyên tố trong $O(\log n)$.

Trace với $N = 12$ – sàng tuyến tính cho ta:

i	2	3	4	5	6	7	8	9	10	11	12
spf[i]	2	3	2	5	2	7	2	3	2	11	2

Để phân tích $x = 12$:

- `spf[12] = 2` → $12 = 2 \cdot 6$.
- `spf[6] = 2` → $6 = 2 \cdot 3$.
- `spf[3] = 3` → $3 = 3 \cdot 1$.
- Dừng. Kết quả: $12 = 2^2 \cdot 3$. Mỗi bước chỉ tốn $O(1)$.

C++

```

int spf[N]; // smallest prime factor
vector<int> primes;

void linear_sieve() {
    for (int i = 2; i < N; i++) {
        if (spf[i] == 0) { // i la so nguyen to
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if ((long long)p * i >= N) break;
            spf[p * i] = p; // diem mau chot: moi so "rot" dung 1 lan
            if (i % p == 0) break;
        }
    }
}

// Phan tich nhanh nho SPF
vector<pair<int, int>> factor(int x) {
    vector<pair<int, int>> res;
    while (x > 1) {
        int p = spf[x], cnt = 0;
        while (x % p == 0) { x /= p; cnt++; }
        res.push_back({p, cnt});
    }
    return res;
}

```

Python

```

spf = [0] * N
primes = []

for i in range(2, N):
    if spf[i] == 0:
        spf[i] = i
        primes.append(i)
    for p in primes:
        if p * i >= N: break
        spf[p * i] = p
        if i % p == 0: break

def factor(x):
    res = []
    while x > 1:
        p = spf[x]; cnt = 0
        while x % p == 0:
            x //= p; cnt += 1
        res.append((p, cnt))
    return res

```

Ưu điểm so với phân tích thử chia $O(\sqrt{n})$: nếu bạn cần phân tích nhiều số $\leq 10^6$ (ví dụ 10^6 số), SPF tổng cộng chỉ $O(n + Q \log n)$ – nhanh hơn hẳn.

7. Bù trừ (Inclusion-Exclusion)

7.1. Vấn đề đặt ra

Đề mẫu: Đếm số nguyên trong $[1, n]$ chia hết cho ít nhất một trong $p_1, p_2, \dots, p_k$ (với p_i là các số nguyên tố phân biệt, k nhỏ).

Ví dụ rất nhỏ: $n = 30, p = \{2, 3\}$. Liệt kê các số chia hết cho 2 hoặc 3 trong $[1, 30]$:

$\{2, 3, 4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20, 21, 22, 24, 26, 27, 28, 30\}$ – đúng 20 số.

Nếu $n = 10^{18}$, không thể liệt kê. Cần công thức.

7.2. Bù trừ cho 2 tập

Gọi A = tập số chia hết cho 2, B = tập số chia hết cho 3. Ta cần $|A \cup B|$.

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Với $n = 30$: $|A| = \lfloor 30/2 \rfloor = 15$, $|B| = \lfloor 30/3 \rfloor = 10$, $|A \cap B| = \lfloor 30/6 \rfloor = 5$ (bội chung là bội của $\text{lcm}(2, 3) = 6$).

$$|A \cup B| = 15 + 10 - 5 = 20$$

7.3. Bù trừ cho 3 tập

Với 3 tập A, B, C , áp dụng tương tự nhưng phức tạp hơn – phần "chung 3" bị trừ 3 lần (qua 3 cặp $A \cap B, A \cap C, B \cap C$) sau khi đã được cộng 3 lần, nên phải cộng bù lại 1 lần:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Ví dụ: $n = 30, p = \{2, 3, 5\}$:

- $|A| = 15, |B| = 10, |C| = 6$
- $|A \cap B| = 5$ (bội 6), $|A \cap C| = 3$ (bội 10), $|B \cap C| = 2$ (bội 15)
- $|A \cap B \cap C| = 1$ (bội 30)
- Tổng: $15 + 10 + 6 - 5 - 3 - 2 + 1 = 22$

7.4. Công thức tổng quát

$$\left| \bigcup_{i=1}^k A_i \right| = \sum_{\emptyset \neq S \subseteq \{1, \dots, k\}} (-1)^{|S|+1} \cdot \left| \bigcap_{i \in S} A_i \right|$$

Tập con cỡ lẻ – dấu cộng; cỡ chẵn – dấu trừ.

Vì p_i là số nguyên tố phân biệt, $|A_{i_1} \cap \dots \cap A_{i_t}| = \lfloor n / (p_{i_1} p_{i_2} \dots p_{i_t}) \rfloor$.

7.5. Cài đặt bằng bitmask

Với k nhỏ (≤ 20), duyệt 2^k tập con bằng bitmask: mỗi mask (từ 1 đến $2^k - 1$) biểu diễn tập con $\{i : \text{bit}_i(\text{mask}) = 1\}$; dấu là + nếu số bit lẻ, – nếu chẵn.

C++

```
int k = p.size();
long long ans = 0;

for (int mask = 1; mask < (1 << k); mask++) {
    long long prod = 1;
    int bits = __builtin_popcount(mask);
    bool overflow = false;
    for (int i = 0; i < k; i++) {
        if (mask >> i & 1) {
            if (prod > n / p[i]) { overflow = true; break; }
            prod *= p[i];
        }
    }
    if (overflow) continue;
    long long term = n / prod;
    ans += (bits % 2 == 1) ? term : -term;
}
return ans;
```

Python

```
def count_multiples(n, p):
    k = len(p)
    ans = 0
    for mask in range(1, 1 << k):
        prod = 1; bits = 0
        for i in range(k):
            if mask >> i & 1:
                prod *= p[i]; bits += 1
                if prod > n: prod = n + 1; break # tránh overflow /
                tích qua lon
        term = n // prod if prod <= n else 0
        ans += term if bits % 2 == 1 else -term
    return ans
```

Trace tay với $n = 30$, $p = \{2, 3\}$ ($k = 2$):

Mask (nhị phân)	Bits	Tập con S	Tích prod	$\lfloor n/\text{prod} \rfloor$	Đóng góp vào ans
01	1	$\{2\}$	2	15	+15
10	1	$\{3\}$	3	10	+10
11	2	$\{2, 3\}$	6	5	–5

Tổng: $15 + 10 - 5 = 20$ – khớp với cách liệt kê ở 7.1.

Khái quát: bù trừ là kỹ thuật "đếm có chồng chéo" – xuất hiện trong rất nhiều bài đếm (đếm hoán vị thỏa điều kiện, đếm bộ số nguyên tố cùng nhau, ...).

7.6. Ứng dụng kinh điển: Đếm hoán vị không bất động (derangement)

Đề: Đếm số hoán vị π của $\{1, 2, \dots, n\}$ sao cho $\pi(i) \neq i$ với mọi i (không có "điểm bất động"). Gọi số đó là D_n .

Áp dụng IE: Gọi A_i = tập hoán vị có $\pi(i) = i$. Ta cần $n! - |A_1 \cup A_2 \cup \dots \cup A_n|$.

Với một tập con S cỡ t , $|\bigcap_{i \in S} A_i| = (n-t)!$ (cố định t điểm bất động, hoán vị tự do $n-t$ điểm còn lại). Số tập con cỡ t là $C(n, t)$. Áp dụng IE:

$$D_n = n! - \sum_{t=1}^n (-1)^{t+1} C(n, t) (n-t)! = n! \sum_{t=0}^n \frac{(-1)^t}{t!}$$

Code: tính trực tiếp tổng trên, hoặc dùng truy hồi $D_n = (n-1)(D_{n-1} + D_{n-2})$.

8. Mẫu kết hợp: đếm dãy có k phần tử "đặc biệt"

Bài toán: Có bảng chữ cái với $|\Sigma| = s$ ký tự. Một ký tự được coi là "đặc biệt". Đếm số dãy độ dài n sao cho có đúng k vị trí mang ký tự đặc biệt.

Phân tích:

- Chọn k vị trí (trong n vị trí) để đặt ký tự đặc biệt: $C(n, k)$ cách.
- Mỗi vị trí trong $n-k$ vị trí còn lại có $s-1$ lựa chọn (mọi ký tự trừ ký tự đặc biệt): $(s-1)^{n-k}$ cách.

Đáp án: $C(n, k) \cdot (s-1)^{n-k} \bmod p$.

Ví dụ cụ thể với $n = 4$, $k = 2$, $s = 3$ (bảng chữ $\{a, b, c\}$, đặc biệt = a):

- $C(4, 2) = 6$ cách chọn vị trí cho a.
- $(3-1)^{4-2} = 2^2 = 4$ cách điền b, c vào 2 vị trí còn lại.

Tổng: $6 \cdot 4 = 24$ dãy.

$C(n, k)$ lấy từ tiền tính (mục 5), $(s-1)^{n-k}$ tính bằng lũy thừa nhanh $\rightarrow$ tổng cộng $O(\log p)$ / truy vấn. Đây là "form" gặp liên tục ở các bài đếm.

9. Bẫy thường gặp

- Quên chuẩn hoá sau phép trừ: dùng $((a - b) \% p + p) \% p$.
- Tràn long long khi nhân: nếu $p \approx 10^9$, hai số $< p$ nhân lại $< 10^{18}$ – vẫn vừa long long nhưng sát ngưỡng; cẩn thận với các bước cộng dồn.
- Áp dụng Fermat khi p không nguyên tố: sai. Khi đó cần extended Euclid (khoá Vàng) hoặc tránh chia.
- Tính $C(n, k)$ khi $k > n$ hoặc $k < 0$ phải trả 0.

- Sàng cho n quá lớn: $N = 10^7$ sàng tuyến tính chiếm ~ 40 MB cho `int` array – coi giới hạn bộ nhớ.
- Bù trừ với quá nhiều tập ($k > 25$): 2^k bùng nổ – phải đổi cách (DP, Möbius).

10. Tổng kết

CÔNG CỤ	KHI DÙNG	CHI PHÍ
Modular inverse (Fermat)	Chia mod p nguyên tố	$O(\log p)$
Precompute $C(n, k)$	Cần nhiều C trong cùng bài	$O(n)$ tiền $+O(1)$ truy vấn
Linear sieve + SPF	Phân tích nhiều số	$O(n)$ sàng $+O(\log x)$ / số
Inclusion-Exclusion	Đếm "ít nhất 1 trong k "	$O(2^k)$